KONERU LAKSHMAIAH EDUCATION FOUNDATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AI FRAMEWORK FOR SOLAR POWER GENERATION FORECASTING

A Mini Project Report Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY
in Computer Science and Engineering

Submitted by

KARTHIKEYA – 2420030616

HARSHITH – 2420090028

NEERAJ – 2420030197

SRIKAR – 2420090025

Academic Year 2026–2027

K L (Deemed to be) University
Hyderabad, Telangana, India

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We hereby declare that the project report entitled “AI Framework for Solar Power Generation Forecasting” is prepared by our team as part of the academic requirements for the Bachelor of Technology degree. This report describes the supplied SolarCast live weather-based solar forecasting application, its design, operating workflow, assumptions, and limitations. The explanation has been prepared with reference to the project files provided by the team.

We confirm that the report distinguishes features visible in the supplied source code from proposed future improvements. In particular, the current application estimates solar power from live weather and solar-radiation data using a formula; it does not contain a trained machine-learning model in the supplied ZIP. We have therefore avoided presenting simulated or invented machine-learning accuracy scores as completed results.

Any external data service and technical references used in this report are acknowledged in the References section. The report is submitted for academic evaluation and is not intended to claim that a production-grade solar plant forecasting system has already been validated.

KARTHIKEYA – 2420030616

HARSHITH – 2420090028

NEERAJ – 2420030197

SRIKAR – 2420090025

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini-project report entitled “AI Framework for Solar Power Generation Forecasting” has been prepared by the following students as part of the academic requirements for the Bachelor of Technology degree in Computer Science and Engineering.

KARTHIKEYA – 2420030616

HARSHITH – 2420090028

NEERAJ – 2420030197

SRIKAR – 2420090025

The report documents the SolarCast live weather-based solar generation estimation application supplied with the project submission. It covers the purpose of the system, software requirements, implementation workflow, formula-based estimation, output presentation, current limitations, and possible future enhancements.

Signature of the Project Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us during the planning, development, and documentation of this mini project. Completing a software project involves more than writing code: it requires identifying a useful problem, understanding available information, testing assumptions, improving the interface, and explaining the outcome clearly. The encouragement and academic environment provided to us helped us approach these tasks with patience and responsibility.

We are thankful to our project guide and the faculty members of the Department of Computer Science and Engineering for their guidance and for encouraging us to connect classroom concepts with a practical application. Their support helped us understand how software design, data handling, interface development, and technical writing contribute to a complete project.

We also acknowledge the university for providing the learning environment and facilities needed to explore web technologies and renewable-energy applications. We appreciate the help of our classmates and friends who discussed ideas, reviewed our progress, and encouraged us to keep improving the project.

We thank our families for their continued encouragement and understanding throughout our academic work. Their support gave us the confidence to complete the application and prepare this report.

Finally, we acknowledge the Open-Meteo service, whose forecast API is used by the supplied application to retrieve weather variables and shortwave-radiation estimates. We have documented this dependency and the limitations of using forecast data as an input to a simplified solar-power calculation. We accept responsibility for the accuracy of the descriptions in this report and for making clear which features are implemented and which remain future work.

KARTHIKEYA – 2420030616
HARSHITH – 2420090028
NEERAJ – 2420030197
SRIKAR – 2420090025

TABLE OF CONTENTS

S. No.	Contents	Page
1	1. ABSTRACT	6–7
2	2. INTRODUCTION	8–11
3	3. LITERATURE SURVEY	12–15
4	4. PROBLEM STATEMENT AND OBJECTIVES	16–18
5	5. HARDWARE AND SOFTWARE REQUIREMENTS	19–20
6	6. IMPLEMENTATION	21–27
7	7. RESULTS AND DISCUSSION	28–30
8	8. CONCLUSION AND FUTURE SCOPE	31
9	9. REFERENCES	32

Note: page ranges are planned for this PDF edition. If the document is reflowed in another editor, page numbering may change.

Project identity: SolarCast live weather-based solar estimation interface. The title uses the requested wording, while the implementation chapter describes the supplied project accurately.

1. ABSTRACT

Solar energy is an important renewable source because it can generate electricity without the direct combustion of fossil fuels during operation. Yet the amount of electricity that a photovoltaic (PV) system can produce changes throughout the day. Sunlight intensity, cloud cover, temperature, the installed system capacity, and the efficiency of the equipment all influence the final output. A useful forecast interface can help a user understand how weather conditions may affect expected generation and plan energy use more thoughtfully.

The project titled “AI Framework for Solar Power Generation Forecasting” is represented by the supplied SolarCast live weather-based solar estimation application. The interface allows a user to specify a location through latitude and longitude, enter a system capacity, and select an assumed system-efficiency percentage. It then requests hourly and daily forecast variables from the Open-Meteo forecast API. The retrieved information includes air temperature, cloud cover, shortwave radiation, daily maximum temperature, daily shortwave-radiation totals, and weather codes.

The current implementation calculates an estimated power value for each hourly forecast point. Its main equation scales installed capacity by solar radiation expressed relative to 1,000 W/m², the selected efficiency, and a temperature adjustment factor. The interface displays estimated daily energy, peak estimated power, daily radiation, a chart for the current day, and a seven-day summary table. It also supports downloading forecast records as a CSV file.

1. ABSTRACT

The project is designed as a practical demonstration of how external weather data can be transformed into a simple solar-generation estimate and presented in a browser-based dashboard. The user can change location, installed capacity, and efficiency to explore different scenarios. A Hyderabad shortcut fills in example coordinates and default system settings. The interface includes status feedback when data is loading or unavailable, which helps users distinguish a completed update from a failed network request.

An important technical distinction is necessary for an academically accurate report. Although the requested project title uses the phrase “AI Framework,” the supplied ZIP contains HTML, CSS, and JavaScript files and calls a public weather API. The code does not include a training dataset, a fitted machine-learning model, model weights, or an evaluation pipeline. Therefore, the delivered implementation is best described as a live weather-based estimation framework and a foundation for a later AI model, rather than a completed trained AI predictor. This report preserves the requested title while describing the implementation as it exists.

The main contribution is an accessible end-to-end prototype: user input, live data retrieval, formula-based estimation, visual presentation, and CSV export. The main limitation is that a simplified formula cannot fully represent a real PV plant, inverter losses, shading, panel orientation, degradation, local calibration, or site-specific operational behaviour. Future development can pair historical generation measurements with weather features and train a forecasting model, then compare that model against the current formula baseline using reproducible metrics.

Keywords: solar power forecasting, photovoltaic generation, shortwave radiation, weather API, energy estimation, JavaScript dashboard, renewable energy.

2. INTRODUCTION

Energy demand continues to grow as homes, businesses, public services, and industries depend on electricity for daily activity. Solar photovoltaic systems are increasingly used because sunlight is widely available and PV systems can be installed at different scales, from rooftop systems to utility-scale plants. Unlike a conventional generator whose fuel input can be controlled directly, solar generation depends strongly on the environmental conditions at the installation site.

Forecasting expected solar generation is useful for planning. A household may want to understand the likely output from a rooftop array, a facility manager may want to compare expected generation with demand, and an operator may need to anticipate periods of lower renewable supply. Forecasts are not guarantees; they are estimates based on available information and assumptions. Their usefulness depends on the quality of the weather input, the model used, and how clearly uncertainty is communicated.

In practical terms, the project follows a straightforward cycle: the user configures the site and system, the application fetches weather data, the data is transformed into estimated power and energy values, and the dashboard communicates the results. This flow makes the prototype suitable for demonstration and further development.

2. INTRODUCTION

The supplied SolarCast application approaches this problem through a browser-based interface. It accepts a location name, latitude, longitude, installed capacity in kilowatts, and a system-efficiency percentage. The application uses the latitude and longitude to request forecast data from Open-Meteo. It requests hourly temperature at two metres, cloud cover, and shortwave radiation, along with daily maximum temperature, daily radiation totals, and weather codes for a seven-day horizon.

After the response arrives, the JavaScript code calculates an estimated output for each hourly record. The calculations are displayed through summary metrics and a chart. A daily table gives the date, a readable weather label, maximum temperature, radiation in kWh/m², and estimated energy in kWh. A CSV export option allows the user to retain the retrieved hourly data and associated estimates for further analysis in a spreadsheet.

In practical terms, the project follows a straightforward cycle: the user configures the site and system, the application fetches weather data, the data is transformed into estimated power and energy values, and the dashboard communicates the results. This flow makes the prototype suitable for demonstration and further development.

2. INTRODUCTION

The title of this academic project refers to an AI framework for solar power generation forecasting. In machine learning, a framework of this kind would normally contain a defined target variable, a historical dataset, data preparation, model training, validation, testing, inference, and evaluation. Those components are not present in the supplied source package. Instead, the current code uses a deterministic engineering-inspired formula, and this report identifies that difference openly rather than inventing an AI model or claiming unmeasured accuracy.

This distinction does not make the prototype unhelpful. A formula-based baseline can be valuable as a first stage: it demonstrates the data flow, provides an initial estimate, gives users an interface for exploring locations and capacity, and establishes a baseline that a later machine-learning model can be compared against. A clear baseline also helps future developers identify which additional information would improve realism.

In practical terms, the project follows a straightforward cycle: the user configures the site and system, the application fetches weather data, the data is transformed into estimated power and energy values, and the dashboard communicates the results. This flow makes the prototype suitable for demonstration and further development.

2. INTRODUCTION

The purpose of this report is to document the project in a clear and human-readable manner. It explains the renewable-energy context, reviews relevant approaches to solar forecasting, identifies the problem and objectives, lists the required hardware and software, and describes the actual implementation. The results chapter focuses on functions observable from the code and on the kinds of outputs the application is designed to show. It does not invent test results that were not supplied.

The report is organised around the same major headings as the reference documentation: Abstract, Introduction, Literature Survey, Problem Statement and Objectives, Hardware and Software Requirements, Implementation, Results and Discussion, Conclusion and Future Scope, and References. This keeps the structure familiar while adapting the content to solar forecasting.

In practical terms, the project follows a straightforward cycle: the user configures the site and system, the application fetches weather data, the data is transformed into estimated power and energy values, and the dashboard communicates the results. This flow makes the prototype suitable for demonstration and further development.

3. LITERATURE SURVEY

Solar generation forecasting sits at the intersection of renewable-energy engineering, weather prediction, statistical modelling, and software visualisation. Existing approaches range from physical models that represent the solar geometry and PV equipment to statistical techniques and machine-learning models that learn relationships from historical observations. The best approach depends on the forecast horizon, the availability of measurements, the required accuracy, and the complexity that the application can support.

Physical and engineering-based approaches use quantities such as irradiance, panel orientation, temperature, module characteristics, and conversion efficiency. These models can be understandable and useful when system details are known. Their limitations arise when site-specific factors are missing or when simplified assumptions are used in place of calibrated equipment characteristics.

Overall, the literature suggests that forecasting quality depends not only on algorithm choice but also on input quality, the forecast horizon, local calibration, and honest evaluation. A model that appears strong on a random split may perform poorly when asked to forecast a genuinely later period, so chronological testing is particularly important for time-dependent solar data.

3. LITERATURE SURVEY

Statistical forecasting methods can model patterns in time series, including daily and seasonal cycles. Regression approaches may relate generation to weather variables such as irradiance, temperature, and cloud cover. More advanced methods can use lagged measurements, calendar features, and weather forecasts. These approaches require suitable historical records and careful evaluation on data that was not used during model fitting.

Machine-learning methods such as random forests, gradient boosting, support vector regression, and neural networks can represent nonlinear relationships between weather and solar generation. Their performance depends on the representativeness of the training data, feature quality, appropriate validation, and the stability of the site and equipment. A model trained on one location may not transfer directly to another without adaptation.

Overall, the literature suggests that forecasting quality depends not only on algorithm choice but also on input quality, the forecast horizon, local calibration, and honest evaluation. A model that appears strong on a random split may perform poorly when asked to forecast a genuinely later period, so chronological testing is particularly important for time-dependent solar data.

3. LITERATURE SURVEY

Deep-learning approaches can model sequences and interactions in large datasets, but they generally require more data, careful tuning, and stronger computing resources. No particular machine-learning algorithm is implemented in the supplied JavaScript project. The current prototype is therefore not described as using random forest, recurrent neural networks, transformers, or any other trained predictor. These algorithms are possible directions for future work, not completed components.

Weather services provide an important input layer for forecasting applications. Open-Meteo exposes forecast variables through a web API, allowing a client application to request values for a chosen latitude and longitude. The supplied code relies on this service to retrieve shortwave radiation and temperature data. As with any forecast provider, the values are model-based estimates and may differ from the conditions measured at a specific solar installation.

Overall, the literature suggests that forecasting quality depends not only on algorithm choice but also on input quality, the forecast horizon, local calibration, and honest evaluation. A model that appears strong on a random split may perform poorly when asked to forecast a genuinely later period, so chronological testing is particularly important for time-dependent solar data.

3. LITERATURE SURVEY

The main research gap addressed by the prototype is practical integration: a user needs a simple way to connect a location and system assumptions to weather-driven output estimates. The dashboard demonstrates this workflow, but a rigorous AI framework would need an additional learning and validation layer. In particular, the future system should compare predictions with measured PV generation, preserve chronological separation between training and test periods, and report errors in units that are meaningful to energy users.

A suitable next phase would evaluate a baseline formula alongside models such as linear regression, random forest regression, gradient boosting, and a neural network. Models should be compared on the same test periods and using metrics such as MAE, RMSE, and possibly normalized error. Any literature claims in the final academic submission should be checked against the original papers and documented consistently.

Overall, the literature suggests that forecasting quality depends not only on algorithm choice but also on input quality, the forecast horizon, local calibration, and honest evaluation. A model that appears strong on a random split may perform poorly when asked to forecast a genuinely later period, so chronological testing is particularly important for time-dependent solar data.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

The problem addressed by this project is to provide a simple, location-aware interface that converts weather forecast information into an understandable estimate of solar power generation. Users should be able to specify basic PV-system assumptions, view the expected hourly profile and daily outlook, and export the underlying forecast data for further inspection.

Problem Description

Solar generation varies because the solar resource changes over time. Cloud cover and atmospheric conditions influence the radiation reaching a panel, while temperature and conversion efficiency affect how much electrical power can be produced from the available sunlight. Without a convenient forecasting interface, a user may need to consult multiple sources, manually interpret weather variables, and perform calculations independently. This can make preliminary planning inconvenient and error-prone.

The supplied application reduces this friction by collecting the relevant weather variables from one API call and presenting them in a consistent dashboard. However, the estimate is a simplified approximation and should not be treated as a guarantee of actual energy output. A fully developed AI solution would require historical generation data and model evaluation, neither of which is included in the supplied ZIP.

Objectives

- Build a browser-based interface for location and PV-system inputs.
- Retrieve hourly temperature, cloud cover, and shortwave radiation from the Open-Meteo forecast API.
- Estimate hourly solar power using installed capacity, radiation, efficiency, and a temperature adjustment.
- Summarise the current-day estimate through daily energy, peak power, and radiation metrics.
- Present a seven-day table with weather labels and daily estimates.
- Provide a chart for viewing the hourly estimated power profile.
- Allow users to export the available hourly forecast and calculated outputs to CSV.
- Handle invalid coordinates and API/network failures with visible feedback.
- Document assumptions and limitations so users understand the difference between estimates and measured generation.
- Establish a baseline architecture that can later be extended with a trained and validated machine-learning model.

4. PROBLEM STATEMENT AND OBJECTIVES

Scope of the Project

The current scope includes a static front end built with HTML, CSS, and JavaScript. The JavaScript validates latitude and longitude, obtains forecast data, applies a simple calculation, updates the dashboard, draws the chart, fills the daily table, and prepares a CSV download. The interface includes a shortcut for Hyderabad using example coordinates, a 5 kW capacity, and an 82 percent efficiency setting.

The project does not include user accounts, a database, a server-side API, a historical data warehouse, a solar-panel physics simulator, a trained AI model, or a formal automated test suite in the supplied archive. These omissions are stated to make the scope precise, not to diminish the work completed in the prototype.

Expected Outcomes

The expected outcome is a working demonstration of the data flow from weather forecast to estimated solar output. With an internet connection, the application can request updated weather information for a selected location. If the service responds with the required fields, the dashboard displays summary values and a daily forecast table. If the request fails, it informs the user that the weather data could not be loaded.

A second expected outcome is a clear basis for future improvement. The formula, data source, user settings, and display logic can be identified and tested independently. A later version can record actual generation, compare it with estimates, tune the assumptions, and replace or supplement the formula with a machine-learning model.

Success Criteria

- The page opens in a modern browser without requiring a build step.
- The form accepts valid geographic coordinates and positive system capacity.
- The application requests the forecast variables required by the calculation.
- The dashboard updates its metrics and table when valid data is returned.
- The chart displays the hourly profile for the selected current day.
- The CSV export contains the hourly weather values and calculated power estimates.
- Errors are surfaced through a status label and readable message rather than silently failing.
- The report accurately distinguishes implemented functionality from proposed AI extensions.

4. PROBLEM STATEMENT AND OBJECTIVES

Assumptions and Constraints

The calculation assumes that the user-supplied installed capacity and efficiency are reasonable representations of the PV system. It also assumes that shortwave radiation from the weather service can serve as a proxy for the radiation available to the installation. Real systems are affected by panel tilt, orientation, shading, soiling, inverter conversion, wiring losses, degradation, and local microclimate, none of which is fully modelled in the current formula.

The hourly energy estimate is produced by summing hourly estimated power values. This is a practical approximation when each value represents a typical power level over an hour, but it should be validated against the intended interpretation of the API's hourly data and against real plant measurements. The application is best suited to education and preliminary exploration, not contractual forecasting or operational dispatch.

Users and Use Cases

A student can use the dashboard to explore the relationship between weather and estimated PV output. A homeowner can test how a different installed capacity changes the rough estimate. A project demonstrator can show how an API, calculation function, chart, and export feature fit together. A researcher could use the exported records as a starting point for offline analysis, provided that the data source, timestamp, units, and assumptions are retained.

For professional use, these scenarios would need stronger data validation, calibrated equipment parameters, clear uncertainty ranges, monitoring, and documented evaluation. The current interface is a prototype and should be presented as such.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- A desktop computer or laptop with a modern multi-core processor.
- At least 4 GB RAM for basic browser use; 8 GB or more is recommended for comfortable development.
- A display and keyboard suitable for editing source files and reviewing the dashboard.
- Enough disk space for the project folder, browser, code editor, and exported CSV files.
- An internet connection to retrieve forecast data from the Open-Meteo API.

A dedicated GPU is not required for the supplied implementation because the application performs straightforward calculations in the browser. A higher-performance computer would become useful if future work adds large historical datasets, model training, or deep-learning experiments.

Software Requirements

Component	Role in the project
HTML	Defines the page structure, controls, metrics, chart area, table, and footer.
CSS	Controls layout, colour, responsive behaviour, spacing, cards, and typography.
JavaScript	Validates inputs, requests weather data, estimates power, renders results, and exports CSV.
Modern web browser	Runs the front-end code and provides fetch, canvas, and download capabilities.
Open-Meteo Forecast API	Provides forecast weather variables and shortwave-radiation values.
VS Code or another editor	Allows the team to inspect and modify project files.
Live Server (optional)	Conveniently serves the static project locally during development.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Development Environment and Dependencies

The supplied archive contains four files: index.html, style.css, script.js, and README.md. This is a lightweight static web project. It does not require a package manager, Python environment, database, or server framework to run the documented features. A developer can extract the archive, open the folder in a code editor, and launch the HTML page in a browser. The README recommends using Live Server or opening index.html directly, while keeping internet access enabled.

The JavaScript uses standard browser facilities including fetch, URLSearchParams, the Canvas 2D API, Blob, object URLs, and programmatic download links. The application calls the Open-Meteo forecast endpoint and builds a query using the entered coordinates, requested hourly and daily fields, a seven-day forecast horizon, and automatic timezone selection.

Network and Data Requirements

The user needs internet access for forecast retrieval. If the network is unavailable, if the API is temporarily unreachable, or if the response does not contain the required hourly radiation series, the page displays an error state. The code also checks that coordinates are within valid geographic ranges and that capacity and efficiency are positive values within expected limits.

The service is external to the project, so availability and returned values are not controlled by the local application. A more robust deployment could include request timeouts, retry limits, caching, API-response logging, and a clear timestamp for every forecast record. The current application displays the update time after a successful fetch and labels Open-Meteo as the data source.

Maintenance and Portability

Because the interface is made of static files, it is relatively easy to share and inspect. A change to the calculation can be made in script.js, while changes to the visual layout belong in style.css and changes to the page structure belong in index.html. This separation makes the project approachable for students and helps the team discuss responsibilities by file.

Before submission, the team should run the project on a clean browser session, confirm the API is reachable, test a valid alternate coordinate, test invalid coordinate values, test the CSV button after loading data, and capture screenshots of the working interface. Screenshots and real runtime observations should be added only after the team has actually verified them.

6. IMPLEMENTATION

6.1 Dataset and Task

The supplied project does not load a local historical solar-generation dataset. Instead, it requests forecast data on demand from the Open-Meteo Forecast API. The task is therefore to estimate near-term solar output from weather forecast variables and user-provided PV assumptions, rather than to train a model from labelled historical examples.

The hourly request includes temperature_2m, cloud_cover, and shortwave_radiation. The daily request includes temperature_2m_max, shortwave_radiation_sum, and weather_code. The request specifies seven forecast days and asks the API to choose the appropriate timezone. The selected latitude and longitude identify the requested location.

6. IMPLEMENTATION

6.2 Data Preprocessing

The code begins by reading form values and converting numeric entries into JavaScript numbers. If the location name is blank, it uses a generic label. The validation function checks that latitude lies between –90 and 90 degrees, longitude lies between –180 and 180 degrees, installed capacity is greater than zero, and efficiency is greater than zero and no greater than one after conversion from a percentage.

After a successful API response, the application verifies that the hourly data and shortwave-radiation array exist. Values are converted to numbers before calculation, and fallback values are supplied for some missing temperature, cloud, or radiation entries. These fallbacks help prevent a missing array element from breaking the display, but a production system should also count missing values and report data quality explicitly.

6. IMPLEMENTATION

6.3 Model Training and Probability Fusion

No model training or probability fusion is implemented in the supplied source code. Instead, the central estimatePower function computes a deterministic value from radiation, temperature, installed capacity, and efficiency. This section retains the reference report's implementation subheading so the document structure remains familiar, but the actual implementation is described accurately.

The current formula is: estimated power in kW = capacity in kW × (shortwave radiation in W/m² ÷ 1000) × efficiency × temperature factor. The temperature factor is set to 1 when temperature is at or below 25°C. Above 25°C, it decreases by 0.004 for each degree above 25°C, with a lower bound of 0.82. The final output is prevented from becoming negative.

6. IMPLEMENTATION

6.4 Explainability

The current application does not include SHAP, LIME, feature-attribution plots, or a trained predictive model to explain. The displayed outputs are interpretable in a simpler way because the calculation is explicit: users can see the capacity and efficiency settings, and the report documents the radiation scaling and temperature adjustment.

For a future AI implementation, explainability could include feature importance, SHAP values, local explanations, and plots comparing actual and predicted generation. Such explanations should be used carefully. They describe relationships learned by a model and do not necessarily prove that a feature causes a change in generation. For the present prototype, the appropriate form of transparency is to disclose the formula, input units, data source, and assumptions.

6. IMPLEMENTATION

6.5 Application/API Workflow

When the page loads, the script updates the capacity and efficiency labels and immediately attempts to load a forecast. When the user submits the settings form or presses the refresh button, the application validates the values, updates the status label, disables the refresh button during the request, and displays a loading message in the daily table.

The script creates the API query, sends a fetch request, checks the HTTP response, parses the returned JSON, and confirms that required radiation data is present. On success it stores the response, renders the metrics and chart, builds the seven-day table, and records the update time and timezone. On failure it switches the status to an error state, shows a readable message, and re-enables the refresh control.

6. IMPLEMENTATION

6.6 Project Workflow

The first stage is configuration. The user enters a location label and coordinates, then chooses installed capacity and system efficiency. The Hyderabad shortcut fills example values and triggers a new request. The second stage is data acquisition, where the browser contacts the forecast API and receives hourly and daily arrays.

The third stage is calculation. The code maps hourly records to time, temperature, radiation, cloud cover, and estimated power. It sums the hourly estimated power values to obtain a daily energy estimate and finds the largest hourly estimate as the peak value. The fourth stage is presentation: the summary cards, canvas chart, and daily table are updated. The final optional stage is export, which creates a CSV file containing the forecast records and derived power estimates.

Stage	Description
Input	Location, coordinates, installed capacity, efficiency
Fetch	Open-Meteo hourly and daily forecast request
Validation	Check coordinate ranges and required radiation data
Estimate	Calculate hourly power with radiation, efficiency, and temperature factor
Display	Update metrics, chart, weather labels, and seven-day table
Export	Create a CSV containing hourly forecast values and estimates

6. IMPLEMENTATION

Implementation details and formula

The function estimatePower first calculates a temperature factor using the selected air temperature. The expression 1 − max(0, temperature − 25) × 0.004 means that temperatures below the reference value do not increase the estimate, while temperatures above it reduce the estimate gradually. Math.max(0.82, ...) prevents this factor from falling below 0.82. The radiation input is clipped at zero before it is divided by 1000 and multiplied by capacity and efficiency.

The formula is deliberately simple. It is not a complete photovoltaic performance model and does not account for panel azimuth, tilt, shading, inverter efficiency curves, DC/AC clipping, soiling, snow, degradation, or detailed thermal behaviour. The result should be called an estimate, not measured or guaranteed generation. Its value is mainly in demonstrating the data pipeline and giving a transparent baseline for future validation.

7. RESULTS AND DISCUSSION

Observed Application Features

The supplied source files establish the following implemented functions: a responsive web interface, editable location and PV settings, a shortcut for Hyderabad, live weather-data retrieval, a formula-based hourly power estimate, current-day summary metrics, a current-day chart, a seven-day forecast table, a CSV download, and visible loading/error feedback. These are implementation findings based on the source code. They are not numerical model-performance results.

The daily energy card is calculated by summing the hourly estimated power values for the selected day. The peak-power card shows the maximum hourly estimate. The radiation card sums the hourly radiation values and divides by 1,000 for its displayed unit. The daily table converts the API's daily shortwave-radiation sum from megajoules per square metre to kilowatt-hours per square metre by dividing by 3.6. It also maps weather codes to labels such as clear, partly cloudy, overcast, fog, rain, snow, showers, or thunderstorm.

Result Interpretation

When the API returns valid data, the dashboard is intended to make the forecast easier to understand than a raw JSON response. Summary cards highlight the scale of the estimated output, the chart shows the hourly pattern, and the table provides context across the forecast horizon. The CSV option allows the user to retain values for a report or spreadsheet. The interface also displays a data-source and update-time label after a successful response.

The project ZIP does not include a captured runtime output, a verified benchmark dataset, or a test report with measured accuracy. Therefore, this report does not claim a particular kWh value for the current day, and it does not provide MAE, RMSE, R^2, accuracy, precision, recall, or ROC-AUC. Those metrics would either be inappropriate for the current formula-only version or would require actual measured-versus-predicted data.

7. RESULTS AND DISCUSSION

Formula and Unit Considerations

The main calculation uses shortwave radiation in watts per square metre and converts it to a 0–1-style scale by dividing by 1,000. It then multiplies this value by installed capacity and the efficiency fraction. This produces a rough power estimate in kilowatts under the chosen assumptions. The temperature adjustment lowers output above 25°C and is bounded to avoid an unlimited decrease.

A unit review is important before the application is used for engineering decisions. The hourly estimate is a power quantity, whereas energy is measured in kilowatt-hours. Summing hourly power values approximates energy only when each power value represents an average or representative value over a one-hour interval. A more rigorous implementation should explicitly integrate power over time, handle irregular timestamps, and compare its estimates against actual meter data.

Limitations and Responsible Use

- The formula is a simplified baseline, not a calibrated PV performance model.
- The project does not contain a trained AI model or historical training dataset.
- Forecast conditions can differ from weather observed at the installation site.
- Installed capacity and efficiency are user-entered assumptions and may be inaccurate.
- Panel orientation, shading, inverter behaviour, soiling, degradation, and outages are not modelled.
- Hourly summation and radiation-unit handling should be validated against real measurements.
- The browser depends on an external API and an active internet connection.
- Forecast values should not be treated as guaranteed generation or used as the sole basis for critical operational decisions.

7. RESULTS AND DISCUSSION

Testing Plan for a Future Evaluation

A repeatable test plan should cover both the user interface and the numerical logic. Interface tests should verify that the page loads, input labels track slider changes, the Hyderabad shortcut fills the expected values, the refresh button shows a loading state, and the table and chart update after a successful API response. Failure tests should include invalid latitude and longitude, an unavailable network, an unsuccessful HTTP response, and a response missing required radiation data.

Numerical tests should isolate estimatePower and verify boundary cases: zero radiation should produce zero output; negative radiation should be clipped to zero; a temperature below 25°C should use a factor of one; a temperature above 25°C should reduce the factor; and the factor should never fall below 0.82. The CSV should be checked for correct headers, escaping of quote characters, and the presence of time, temperature, cloud cover, radiation, and estimated power columns.

Evaluation Plan for an AI Extension

To claim an AI forecasting result, the team would first need a dataset containing timestamped measured solar generation and aligned weather observations or forecasts. The data should be cleaned, timestamps standardised, missing values examined, and features constructed without using information that would not have been available at forecast time. Training and evaluation should be separated chronologically so that later observations remain unseen during model development.

A baseline model should be compared with candidate regressors using the same held-out period. Mean Absolute Error (MAE) measures the average absolute error, Root Mean Squared Error (RMSE) penalises larger errors more strongly, and R^2 describes variance explained relative to a baseline mean. Results should be reported with units, test period, sample count, location, installed capacity, and any exclusions. Only then could the project make evidence-based claims about forecasting performance.

8. CONCLUSION AND FUTURE SCOPE

This project documents a browser-based framework for estimating solar power generation from live weather forecast information. The supplied SolarCast application allows users to choose a location, set an installed capacity and efficiency assumption, request forecast variables, estimate hourly power, view current-day summary metrics, inspect a seven-day table, and export the hourly records to CSV. The design is compact and understandable, making it useful as a student prototype and a starting point for further development.

The most important conclusion is that the delivered version is a formula-based weather estimator rather than a completed trained AI model. The requested project title has been retained, but the report does not claim model training or accuracy scores that are not supported by the source files. This transparent description gives the team a reliable foundation for explaining the implementation during a project review.

Future Scope

- Collect timestamped historical PV generation and weather data for one or more sites.
- Create a reproducible preprocessing pipeline and document the dataset source and licence.
- Train regression models such as linear regression, random forest, gradient boosting, and neural networks.
- Use chronological train, validation, and test splits to prevent temporal leakage.
- Compare models against the current formula baseline using MAE, RMSE, and R^2.
- Add uncertainty intervals and communicate that forecasts are estimates.
- Calibrate the system for panel tilt, orientation, inverter losses, shading, and temperature behaviour.
- Add caching, timeout handling, retries, data-quality checks, and automated tests.
- Store forecast runs and actual measurements to support monitoring and model drift analysis.
- Improve accessibility, mobile layout, and report-ready chart and CSV exports.

With these improvements, the application could evolve from an educational weather-based estimator into a more complete solar-generation forecasting platform. The next milestone should be a verified dataset and a reproducible baseline evaluation before introducing more complex AI methods.

9. REFERENCES

1. Open-Meteo. (n.d.). *Weather Forecast API*. https://open-meteo.com/

2. Open-Meteo. (n.d.). *API documentation*. https://open-meteo.com/en/docs

3. National Renewable Energy Laboratory (NREL). (n.d.). *Photovoltaic research and performance resources*. https://www.nrel.gov/

4. International Energy Agency (IEA). (n.d.). *Solar PV*. https://www.iea.org/energy-system/renewables/solar-pv

5. International Renewable Energy Agency (IRENA). (n.d.). *Renewable power generation costs and renewable energy resources*. https://www.irena.org/

6. MDN Web Docs. (n.d.). *Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

7. MDN Web Docs. (n.d.). *Canvas API*. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

8. MDN Web Docs. (n.d.). *Blob*. https://developer.mozilla.org/en-US/docs/Web/API/Blob

9. MDN Web Docs. (n.d.). *URL.createObjectURL()*. https://developer.mozilla.org/en-US/docs/Web/API/URL/createObjectURL_static

10. Project source package. (2026). *solarcast_live_forecast*: index.html, style.css, script.js, and README.md, supplied by the project team.

Note on sources: the implementation-specific descriptions in this report are based on inspection of the supplied project source files. External references are provided as background reading. Any research paper or dataset added in a later revision should be checked against its original publication and cited with complete bibliographic details.

PROJECT TEAM

KARTHIKEYA – 2420030616

HARSHITH – 2420090028

NEERAJ – 2420030197

SRIKAR – 2420090025